Universidad Andrés Bello

Facultad de Ingeniería
Optimización

Optimización: Modelado, Implementación y Análisis de Problemas NP-Completos
Steiner Tree

Autores:
Vicente Jiménez
Cristóbal Osorio
Nicolas Sandoval

Profesor:
Luis Fuentes Larenas

Santiago, Chile
2026

Universidad Andrés Bello

Índice de contenido

1. Introducción....3
2. Definición del problema....4
2.1. Descripción en lenguaje natural.... 4
2.2. Definición formal (entrada/salida)....5
2.2.1. Entrada.... 5
2.2.2. Salida....6
2.3. Ejemplo ilustrativo.... 6
2.4. Aplicaciones.... 8
2.4.1. Redes de telecomunicaciones....8
2.4.2. Diseño de circuitos electrónicos (VLSI).... 8
2.4.3. Redes de transporte.... 8
2.4.4. Biología computacional.... 8
2.4.5. Distribución de energía.... 8
3. Modelado matemático....9
3.1. Conjuntos y parámetros....9
3.2. Variables de decisión.... 9
3.3. Restricciones....9
3.3.1. Conectividad de terminales....9
3.3.2. Inclusión de terminales.... 10
3.3.3. Consistencia de estructura (árbol)....10
3.3.4. Selección válida de aristas.... 10
3.4. Función objetivo.... 10
3.5. Versión de decisión....10
4. Complejidad y contexto.... 12
4.1. Justificación de pertenencia a NP....12
4.2. Explicación conceptual de NP-completitud.... 12
4.3. Relación con otros problemas....13
5. Implementación....13
5.1. Python....14
5.1.1. Una solución base (fuerza bruta).... 14
5.1.1.1. Componentes clave del código....15
5.1.1.2. Complejidad.... 15
5.1.2. Una solución mejorada (heurística).... 15
5.1.2.1. Componentes clave del código....16
5.1.2.2. Complejidad.... 16
5.2. C++....16
5.2.1. Una solución base (fuerza bruta).... 17
5.2.1.1. Componentes clave del código....17

5.2.1.2. Complejidad..... 18
5.2.2. Una solución mejorada (heurística)..... 18
5.2.2.1. Componentes clave del código..... 18
5.2.2.2. Complejidad..... 19
5.3. Java..... 19
5.3.1. Una solución base (fuerza bruta)..... 19
5.3.1.1. Componentes clave del código..... 20
5.3.1.2. Complejidad..... 20
5.3.2. Una solución mejorada (heurística)..... 20
5.3.2.1. Componentes clave del código..... 21
5.3.2.2. Complejidad..... 21
6. Análisis experimental..... 21
Tabla 6.1. Resultados experimentales por caso y lenguaje..... 22
Tabla 6.2. Instancias utilizadas y calidad de la heurística..... 22
6.1. Tiempo de ejecución..... 22
6.1.1. Análisis del tiempo de ejecución (solución exacta)..... 22
Figura 6.3. La solución exacta crece rápidamente por la enumeración exponencial de subconjuntos. La escala logarítmica permite observar mejor las diferencias..... 23
6.1.2. Análisis del tiempo de ejecución (heurística)..... 23
Figura 6.4.. La heurística mantiene tiempos bajos porque evita explorar todos los subconjuntos de nodos opcionales..... 24
Figura 6.5. Comparación entre costo óptimo y costo heurístico..... 25
6.2. Escalabilidad..... 25
6.3. Diferencias entre enfoques..... 26
6.4. Diferencias entre lenguajes..... 26
7. Conclusión..... 26
8. Bibliografía..... 27
9. Anexo..... 27

1. Introducción

Este informe corresponde a un trabajo de investigación en el área de la teoría de la complejidad computacional, cuyo objetivo es estudiar en profundidad uno de los 21 problemas NP-completos clásicos identificados por Richard Karp. Estos problemas constituyen una base fundamental para comprender los límites de la computación eficiente, así como las dificultades asociadas al diseño de algoritmos para problemas de alta complejidad.

El problema seleccionado para el desarrollo de este informe es el **Steiner Tree**, el cual será analizado desde un enfoque conceptual, matemático y computacional. En particular, se abordará su definición, modelado y complejidad, junto con distintas estrategias de resolución.

Además, se implementarán soluciones en Python, C++ y Java, considerando tanto un enfoque base (fuerza bruta o backtracking) como una versión mejorada (heurística o técnica de optimización). Finalmente, se realizará un análisis experimental comparando el rendimiento, tiempo de ejecución y escalabilidad entre los distintos enfoques y lenguajes.

2. Definición del problema

El problema de Steiner Tree es uno de los problemas clásicos de optimización combinatoria y forma parte del conjunto de problemas NP-completos identificados por Richard Karp en su influyente trabajo de 1972 sobre la complejidad computacional.

A diferencia del problema del árbol generador mínimo (Minimum Spanning Tree), el problema de Steiner permite la inclusión de nodos adicionales —denominados *nodos de Steiner*— que no pertenecen al conjunto original de nodos requeridos. Estos nodos no vienen dados en la entrada del problema, sino que son incorporados estratégicamente como parte de la solución con el objetivo de reducir el costo total de conexión. En términos intuitivos, actúan como puntos intermedios que permiten construir rutas más eficientes entre los nodos terminales, evitando conexiones directas que podrían resultar más costosas. De esta manera, el árbol resultante no se limita a los nodos originales, sino que se "expande" mediante la adición de estos nodos auxiliares, generando estructuras más óptimas que las que se obtendrían si solo se consideran los nodos iniciales. Sin embargo, esta flexibilidad incrementa significativamente el espacio de búsqueda, ya que no solo se debe decidir cómo conectar los nodos, sino también qué nodos adicionales conviene incluir, lo que introduce una complejidad combinatoria considerable.

2.1. Descripción en lenguaje natural

El problema de **Steiner Tree** consiste en encontrar la forma más económica de conectar un conjunto específico de nodos dentro de un grafo ponderado. Estos nodos obligatorios se denominan *terminales*, y deben quedar todos conectados entre sí mediante un subgrafo.

A diferencia de otros problemas de conectividad, aquí se permite la inclusión de nodos adicionales no requeridos originalmente (nodos de Steiner). Estos nodos intermedios no forman parte del conjunto objetivo, pero pueden ser incorporados si ayudan a reducir el costo total de conexión. En la práctica, funcionan como "puntos de apoyo" que permiten construir rutas más cortas o más baratas entre los terminales.

El problema, por tanto, no solo consiste en decidir cómo conectar los nodos terminales, sino también en determinar si conviene o no incluir nodos

intermedios para mejorar la solución, lo que aumenta considerablemente la cantidad de configuraciones posibles.

2.2. Definición formal (entrada/salida)

Sea un grafo no dirigido y ponderado:

$$\mathbf{G = (V , E)}$$

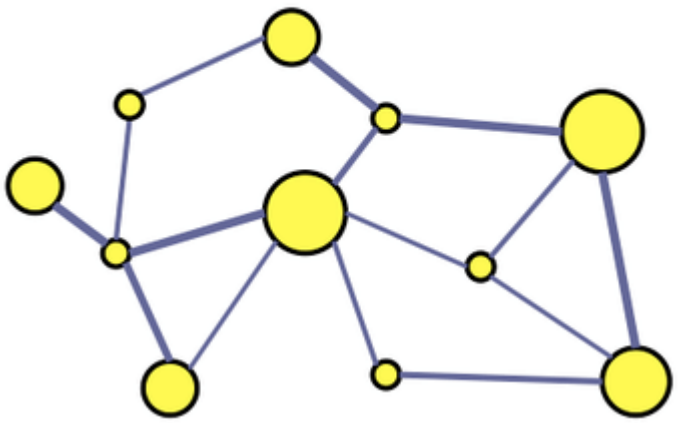
donde:
V : conjunto de nodos
E : conjunto de aristas
Cada arista $\mathbf{e} \in \mathbf{E}$ tiene un costo $\mathbf{c(e) \geq 0}$

Sea además:
$\mathbf{T} \subseteq \mathbf{V}$: conjunto de nodos terminales

2.2.1. Entrada

Se recibe un grafo no dirigido y ponderado G = (V , E) , junto con una función de costos c: E → R +, y un subconjunto de nodos terminales $\mathbf{T} \subseteq \mathbf{V}$ **T** que deben ser conectados.

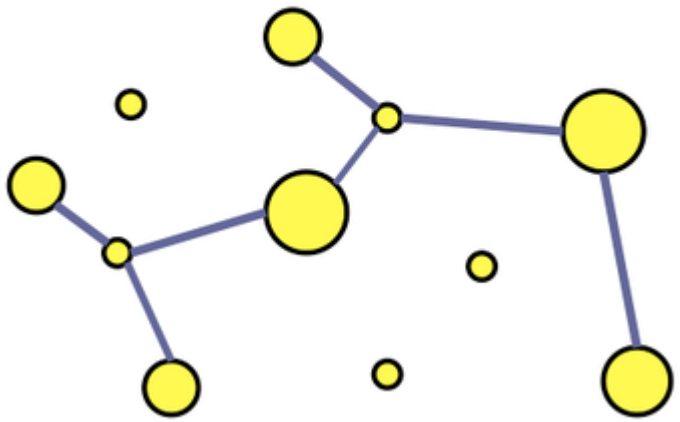
A) Input Graph, G = (V, E)



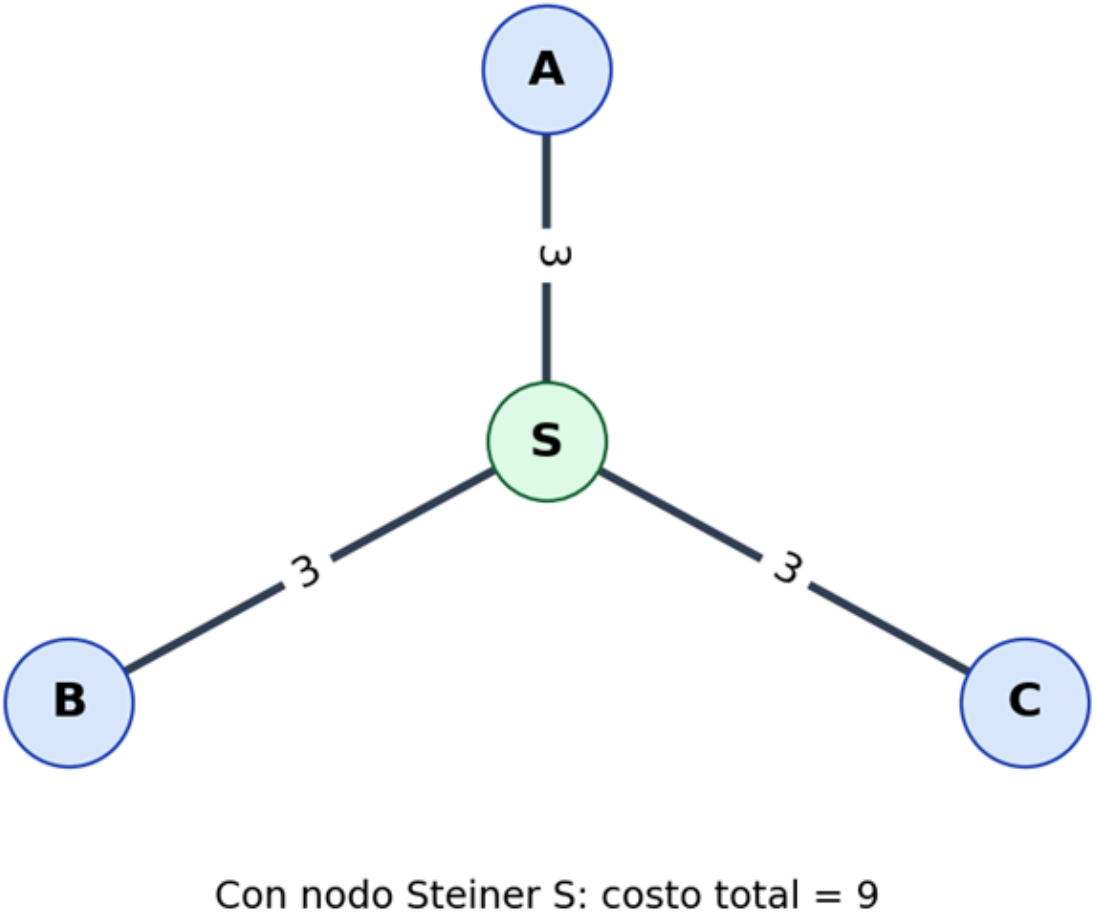
2.2.2. Salida

Se busca un subgrafo conexo **G′ = (V′ , E′)** , tal que incluya todos los nodos terminales **(T ⊆ V′ T)** y cuyo costo total de aristas sea mínimo.

B) Prize-collecting Steiner Tree, G’ = (V’, E’)



2.3. Ejemplo ilustrativo



Para comprender mejor el problema, consideremos un caso sencillo con tres nodos terminales **A**, **B** y **C**. Si estos nodos se conectan directamente entre sí, se puede formar un triángulo donde cada arista tiene un costo asociado. En este caso, la conexión directa entre todos los nodos genera un costo total relativamente alto.

Sin embargo, si se permite la inclusión de un nodo adicional **S** (nodo de Steiner), ubicado estratégicamente en el centro, es posible conectar **A**, B y **C** a través de este nodo intermedio. De esta forma, las distancias individuales se reducen, y el costo total de conexión puede ser menor que el obtenido conectando directamente los terminales entre sí.

Este ejemplo ilustra la idea central del problema: la introducción de nodos no requeridos puede mejorar significativamente la solución final, reduciendo el costo global de la red.

2.4. Aplicaciones

El problema de Steiner Tree tiene múltiples aplicaciones prácticas en distintos campos donde se busca optimizar conexiones:

2.4.1. Redes de telecomunicaciones

Diseño de redes de fibra óptica minimizando el costo de infraestructura.

2.4.2. Diseño de circuitos electrónicos (VLSI)

Optimización de conexiones entre componentes dentro de microchips.

2.4.3. Redes de transporte

Planificación de rutas eficientes entre ciudades o puntos logísticos.

2.4.4. Biología computacional

Reconstrucción de árboles evolutivos y filogenéticos.

2.4.5. Distribución de energía

Diseño de redes eléctricas eficientes con menor costo de tendido.

En todos estos casos, el objetivo común es reducir el costo total de conexión mientras se asegura la conectividad entre los elementos esenciales del sistema.

3. Modelado matemático

El problema de Steiner Tree puede formularse como un problema de optimización combinatoria sobre grafos, donde se busca seleccionar un subconjunto de aristas que conecte todos los nodos terminales con el menor costo posible, permitiendo la inclusión de nodos intermedios.

3.1. Conjuntos y parámetros

Sea:

- **G = (V , E)** : grafo no dirigido y ponderado
- **T** $\subseteq$ **V T** : conjunto de nodos terminales
- **c (e)** : costo asociado a cada arista **e** $\in$ **E**

3.2. Variables de decisión

Se define una variable binaria:

$$x_e = \begin{cases} 1 & \text{si la arista } e \text{ pertenece a la solución} \\ 0 & \text{en caso contrario} \end{cases}$$

3.3. Restricciones

3.3.1. Conectividad de terminales

Todos los nodos terminales deben estar conectados dentro del subgrafo inducido por las aristas seleccionadas.

3.3.2. Inclusión de terminales

Se debe garantizar que todos los nodos del conjunto **T** estén presentes en la solución final:

$$\mathbf{T} \subseteq \mathbf{V'}\ \mathbf{T}$$

3.3.3. Consistencia de estructura (árbol)

El subgrafo resultante debe ser conexo y acíclico (estructura tipo árbol), lo que implica evitar ciclos innecesarios que incrementen el costo.

3.3.4. Selección válida de aristas

Las aristas seleccionadas deben inducir una estructura válida dentro del grafo original.

3.4. Función objetivo

El objetivo es minimizar el costo total de las aristas seleccionadas:

$$\min \sum_{e \in E} c(e)\, x_e$$

3.5. Versión de decisión

La versión de decisión del problema de Steiner Tree se formula de la siguiente manera:

Dado un grafo no dirigido y ponderado **G = (V , E)** , un conjunto de terminales **T** $\subseteq$ **V** y un valor umbral K, se plantea la siguiente pregunta:

¿Existe un subgrafo conexo que conecte todos los nodos de T y cuyo costo total sea menor o igual a K?

Formalmente, existe un subgrafo **G ′ = (V′ , E′)** tal que:

- $\mathbf{T} \subseteq \mathbf{V'} \subseteq \mathbf{V}$
- $\mathbf{G'}$ es conexo
- $$\sum_{e \in E'} c(e) \leq K$$

Esta formulación es especialmente relevante en teoría de complejidad, ya que transforma el problema de optimización en un problema de decisión, lo cual es fundamental para demostrar que el Steiner Tree pertenece a la clase NP-completo

4. Complejidad y contexto

4.1. Justificación de pertenencia a NP

El problema de Steiner Tree pertenece a la clase NP porque, dada una solución candidata (un subgrafo), es posible verificar en tiempo polinomial si dicha solución es válida.

En particular, se puede comprobar eficientemente que:

- Todos los nodos terminales están conectados.
- El subgrafo es conexo.
- El costo total de las aristas no supera un valor dado (en la versión de decisión).

Esto significa que, aunque encontrar la solución óptima puede ser muy costoso, verificar una solución propuesta sí es eficiente.

4.2. Explicación conceptual de NP-completitud

Un problema es NP-completo si pertenece a la clase NP y, además, es tan difícil como cualquier otro problema en NP. Esto implica que no se conoce un algoritmo que lo resuelva en tiempo polinomial para todos los casos, y se cree que dicho algoritmo no existe.

En el caso del **Steiner Tree**, su NP-completitud se establece mediante reducciones desde problemas conocidos como Set Cover. Esto significa que cualquier instancia de estos problemas puede transformarse en una instancia

de Steiner Tree en tiempo polinomial, por lo que resolver eficientemente Steiner Tree permitiría resolver también otros problemas NP-completos.

De forma intuitiva, la dificultad del problema radica en la gran cantidad de combinaciones posibles que deben evaluarse, ya que no solo se deben seleccionar las aristas que conectan los nodos, sino también decidir qué nodos adicionales incluir para minimizar el costo total. Esto genera un crecimiento exponencial en el número de soluciones posibles.

4.3. Relación con otros problemas

El Steiner Tree está fuertemente relacionado con otros problemas clásicos de optimización:

- Es una generalización del árbol generador mínimo (MST), donde los nodos terminales no son todos los nodos del grafo.

- Se relaciona con problemas de conectividad como:

    - Minimum Spanning Tree

    - Traveling Salesman Problem

- Comparte técnicas de aproximación con problemas de cobertura y selección de conjuntos.

5. Implementación

En esta sección se desarrollan distintas estrategias para resolver el problema del Steiner Tree, considerando tanto enfoques exactos como aproximados. Además, se implementan soluciones en tres lenguajes de programación: Python, C++ y Java, con el objetivo de comparar su rendimiento.

Para cada lenguaje, se consideran dos enfoques:

- Una solución base (fuerza bruta)
- Una solución mejorada (heurística)

No se utilizaron librerías que resuelvan directamente el problema.

5.1. Python

En esta sección se presenta la implementación del problema de **Steiner Tree** en Python, considerando dos enfoques: uno exacto basado en enumeración exhaustiva y otro aproximado basado en una heurística eficiente.

5.1.1. Una solución base (fuerza bruta)

La solución exacta implementada se basa en la enumeración de subconjuntos de nodos de Steiner. La idea principal consiste en considerar todos los posibles subconjuntos de nodos no terminales y, para cada uno de ellos, construir un árbol generador mínimo (MST) sobre el conjunto formado por los nodos terminales más los nodos seleccionados.

Para cada subconjunto:

- Se construye el conjunto de nodos candidatos (terminales + nodos adicionales).

- Se calcula el costo del árbol generador mínimo inducido sobre dichos nodos.

- Se mantiene el costo mínimo encontrado.

5.1.1.1. Componentes clave del código

- **Enumeración de subconjuntos:** uso de máscaras binarias para recorrer todas las combinaciones posibles de nodos no terminales.
- **Estructura DSU (Disjoint Set Union):** utilizada para implementar eficientemente el algoritmo de Kruskal.
- **Función mst_cost_induced:** calcula el costo del MST restringido a un subconjunto de nodos.
- **Validación de conectividad:** asegura que todos los nodos seleccionados estén conectados.

5.1.1.2. Complejidad

La complejidad del algoritmo es exponencial:

$$\mathbf{O(2^{|V \setminus T|} \cdot E \log E)}$$

por lo que solo es viable para instancias pequeñas

5.1.2. Una solución mejorada (heurística)

La solución heurística implementada se basa en la técnica de clausura métrica + árbol generador mínimo, que aproxima el problema de Steiner Tree de la siguiente manera:

- Se ejecuta Dijkstra desde cada nodo terminal para obtener distancias mínimas hacia todos los nodos.

- Se construye un grafo completo entre los terminales usando estas distancias.

- Se calcula un MST sobre este grafo reducido.

- Se reconstruyen los caminos correspondientes en el grafo original.

- Finalmente, se calcula un MST sobre la unión de estos caminos para eliminar ciclos.

5.1.2.1. Componentes clave del código

- **Algoritmo de Dijkstra:** cálculo de caminos mínimos desde cada terminal.
- **Construcción de clausura métrica:** distancias entre pares de terminales.
- **DSU:** para construir el MST sobre los terminales.
- **Reconstrucción de caminos:** uso de arreglos parent y parent_edge.
- **Optimización final:** eliminación de ciclos mediante un MST adicional.

5.1.2.2. Complejidad

$$O(k \cdot (E\log V)+k2\log k)$$

donde k es el número de terminales.

5.2. C++

La implementación en C++ sigue el mismo enfoque desarrollado en Python, incorporando tanto una solución exacta basada en enumeración exhaustiva como una solución heurística basada en clausura métrica y árboles generadores mínimos. Sin embargo, se aprovechan las características del lenguaje para mejorar el rendimiento y el control de memoria.

5.2.1. Una solución base (fuerza bruta)

La solución exacta en C + + sigue la estrategia de enumeración de subconjuntos de nodos de Steiner. Se consideran todos los posibles subconjuntos de nodos no terminales y, para cada uno, se construye un árbol generador mínimo (MST) sobre el conjunto formado por los nodos terminales y los nodos adicionales seleccionados.

El proceso consiste en:

- Identificar los nodos no terminales.
- Generar todas las combinaciones posibles mediante máscaras binarias.
- Construir el conjunto de nodos seleccionados.
- Calcular el costo del MST inducido utilizando el algoritmo de Kruskal.
- Mantener el mínimo costo encontrado.

5.2.1.1. Componentes clave del código

- **Máscaras binarias (mask):** permiten generar todos los subconjuntos de nodos opcionales.
- **Estructura DSU (Disjoint Set Union):** utilizada para implementar Kruskal de forma eficiente.

- Función mstCostInduced: calcula el costo del MST sobre los nodos seleccionados.
- **Uso de vector <char>:** optimiza la verificación de pertenencia de nodos.

- **Ordenamiento de aristas:** necesario para aplicar Kruskal correctamente.

5.2.1.2. Complejidad

La complejidad del algoritmo es:

$$O(2\,|V \setminus T|\cdot E\log E)$$

lo que lo hace inviable para grafos grandes, pero adecuado para validar soluciones óptimas en instancias pequeñas.

5.2.2. Una solución mejorada (heurística)

La solución heurística en C++ implementa una aproximación basada en clausura métrica + MST, siguiendo los mismos pasos que en Python:

- Ejecutar Dijkstra desde cada nodo terminal.

- Construir un grafo completo entre terminales usando distancias mínimas.

- Aplicar Kruskal para obtener un MST sobre los terminales.

- Reconstruir los caminos en el grafo original.

- Construir un MST final sobre las aristas utilizadas para eliminar ciclos.

5.2.2.1. Componentes clave del código

- **Algoritmo de Dijkstra:** implementado con priority_queue para eficiencia.

- **Estructura tuple:** utilizada para representar aristas y listas de adyacencia.

- **Clausura métrica (closure):** almacena distancias entre pares de terminales.

- **DSU aplicado a terminales:** construcción del MST sobre el grafo reducido.
- **Reconstrucción de caminos:** mediante arreglos parent y parentEdge.
- **MST final:** elimina ciclos en el subgrafo generado.

5.2.2.2. Complejidad

La complejidad aproximada es:

$$O(k \cdot (E \log V) + k\ 2 \log k)$$

donde k es el número de nodos terminales.

5.3. Java

En esta sección se presenta la implementación del problema del **Steiner Tree** en Java, manteniendo los mismos enfoques utilizados en Python y C++: una solución exacta basada en fuerza bruta y una solución heurística eficiente.

5.3.1. Una solución base (fuerza bruta)

La solución exacta en Java sigue la estrategia de enumeración de subconjuntos de nodos de Steiner. Se consideran todos los subconjuntos posibles de nodos no terminales y, para cada uno, se construye un árbol generador mínimo (MST) sobre el conjunto formado por los nodos terminales y los nodos seleccionados.

El procedimiento consiste en:

- Identificar los nodos no terminales.
- Generar todas las combinaciones mediante máscaras binarias.
- Construir el conjunto de nodos seleccionados.

- Calcular el costo del MST inducido utilizando el algoritmo de Kruskal.

- Mantener el menor costo encontrado.

5.3.1.1. Componentes clave del código

- **Máscaras binarias (mask)**: generación de subconjuntos de nodos opcionales.
- **Clase DSU**: implementación de Union-Find para el algoritmo de Kruskal.
- **Función mstCostInduced**: cálculo del costo del MST restringido.
- **Uso de arreglos booleanos (boolean[])**: para verificar pertenencia de nodos.
- **Ordenamiento de aristas (Comparator)**: necesario para aplicar Kruskal.

5.3.1.2. Complejidad

$$O(2^{|V \setminus T|} \cdot E \log E)$$

Este enfoque es computacionalmente costoso y solo aplicable a instancias pequeñas.

5.3.2. Una solución mejorada (heurística)

La solución heurística implementada en Java utiliza la técnica de **clausura métrica + MST**, siguiendo el mismo esquema que en Python y C++:

- Ejecutar **Dijkstra** desde cada nodo terminal.

- Construir un grafo completo entre terminales usando distancias mínimas.

- Aplicar Kruskal para obtener un MST sobre los terminales.

- Reconstruir los caminos en el grafo original.

- Eliminar ciclos mediante un MST final.

5.3.2.1. Componentes clave del código

- **Algoritmo de Dijkstra**: implementado con PriorityQueue.
- **Clase AdjEdge**: representación eficiente del grafo.
- **Clausura métrica (closure)**: distancias entre terminales.
- **DSU sobre terminales**: construcción del MST reducido.
- **Reconstrucción de caminos**: mediante arreglos parent y parentEdge.
- **MST final**: eliminación de ciclos en el subgrafo resultante.

5.3.2.2. Complejidad

$$O(k \cdot (E \log V) + k\ 2 \log k)$$

donde k es el número de nodos terminales.

6. Análisis experimental

Las ejecuciones se realizaron con las mismas instancias en los tres lenguajes. Cada medición corresponde al tiempo promedio interno del algoritmo después de una ejecución de calentamiento, para reducir el efecto de inicialización.

Caso	Lenguaje	Exacto ms	Heur. ms	Óptimo	Heurística	Brecha %
01_pequeno	C++	0.0048	0.0025	9	10	11.11
01_pequeno	Java	0.1334	0.1807	9	10	11.11
01_pequeno	Python	0.0519	0.0290	9	10	11.11
02_manual	C++	0.0074	0.0041	19	21	10.53
02_manual	Java	0.3703	0.2866	19	21	10.53
02_manual	Python	0.0936	0.0429	19	21	10.53
03_n8	C++	0.0071	0.0041	28	29	3.57
03_n8	Java	0.3478	0.4255	28	29	3.57
03_n8	Python	0.1039	0.0434	28	29	3.57
04_n10	C++	0.0214	0.0046	18	18	0.0
04_n10	Java	0.8914	0.1956	18	18	0.0
04_n10	Python	0.4234	0.0496	18	18	0.0
05_n12	C++	0.0978	0.0049	16	16	0.0
05_n12	Java	1.3659	0.3511	16	16	0.0
05_n12	Python	2.0574	0.0566	16	16	0.0
06_n14	C++	0.3505	0.0071	38	38	0.0
06_n14	Java	2.2768	0.4110	38	38	0.0
06_n14	Python	4.8286	0.0922	38	38	0.0
07_n16	C++	1.3771	0.0081	23	23	0.0
07_n16	Java	10.1440	0.5915	23	23	0.0
07_n16	Python	22.8252	0.1106	23	23	0.0

Tabla 6.1. Resultados experimentales por caso y lenguaje.

Los programas usan el mismo formato para facilitar la comparación entre lenguajes:

```
n m
u v w   # m líneas de aristas no dirigidas, con nodos 0-indexados
k t1 t2 ... tk
```

Caso	n	m	|T|	Óptimo	Heurística	Brecha %
caso_01_pequeno	6	9	3	9	10	11.11
caso_02_manual	8	13	4	19	21	10.53
caso_03_n8	8	14	4	28	29	3.57
caso_04_n10	10	18	4	18	18	0.0
caso_05_n12	12	22	4	16	16	0.0
caso_06_n14	14	26	5	38	38	0.0
caso_07_n16	16	30	5	23	23	0.0

Tabla 6.2. Instancias utilizadas y calidad de la heurística.

La brecha porcentual permite evaluar la calidad de la solución heurística respecto a la solución óptima. Se observa que, en la mayoría de los casos, la heurística logra resultados iguales o muy cercanos al óptimo, lo que demuestra su efectividad práctica, especialmente considerando su menor costo computacional.

6.1. Tiempo de ejecución

6.1.1. Análisis del tiempo de ejecución (solución exacta)

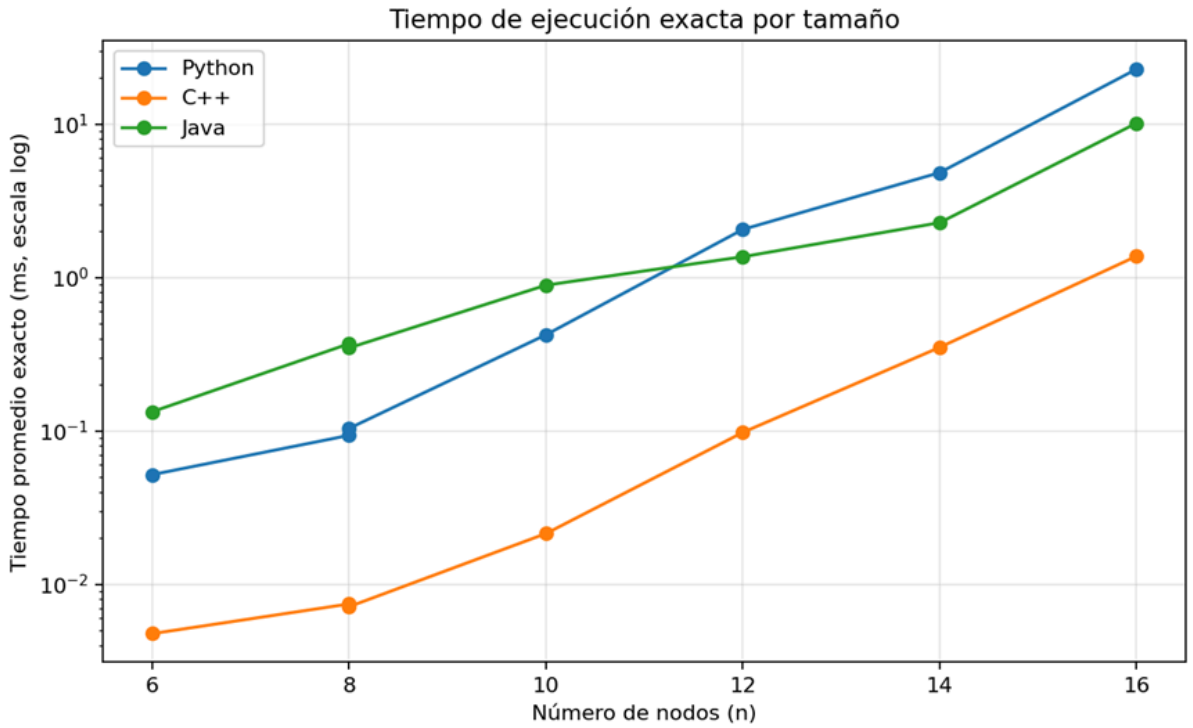


Figura 6.3. La solución exacta crece rápidamente por la enumeración exponencial de subconjuntos. La escala logarítmica permite observar mejor las diferencias.

Los resultados muestran claramente que la solución exacta presenta un crecimiento exponencial en el tiempo de ejecución a medida que aumenta el número de nodos.

- **Python** presenta el peor rendimiento, alcanzando tiempos superiores a 20 ms en instancias de mayor tamaño.
- **Java** muestra un rendimiento intermedio.
- **C + +** es significativamente más eficiente, manteniendo tiempos bajos incluso en instancias más grandes.

Esto confirma que la complejidad teórica del algoritmo exacto se refleja en la práctica, volviéndolo inviable para grafos grandes.

6.1.2. Análisis del tiempo de ejecución (heurística)

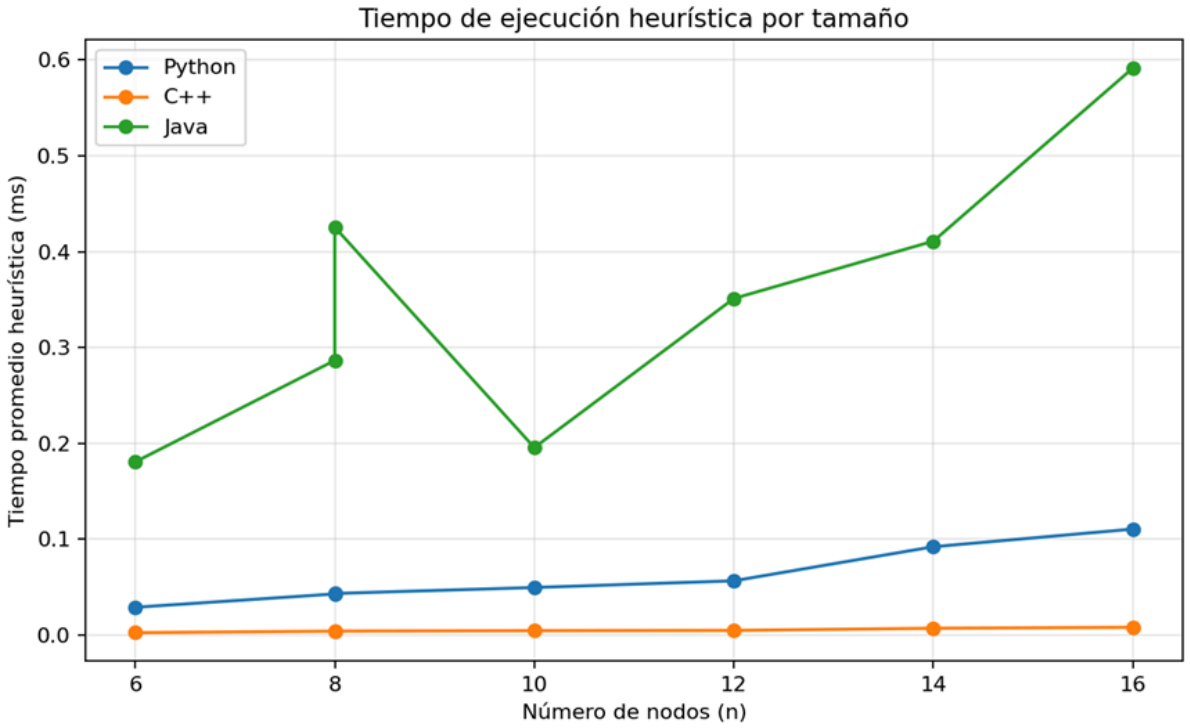


Figura 6.4.. La heurística mantiene tiempos bajos porque evita explorar todos los subconjuntos de nodos opcionales.

En contraste, la solución heurística presenta un crecimiento mucho más controlado:

- Los tiempos se mantienen bajos en todos los lenguajes.
- **C + +** nuevamente muestra el mejor rendimiento.
- **Python** y **Java** presentan tiempos mayores, pero siguen siendo eficientes.

Esto demuestra que la heurística es **altamente escalable**, siendo adecuada para instancias de mayor tamaño.

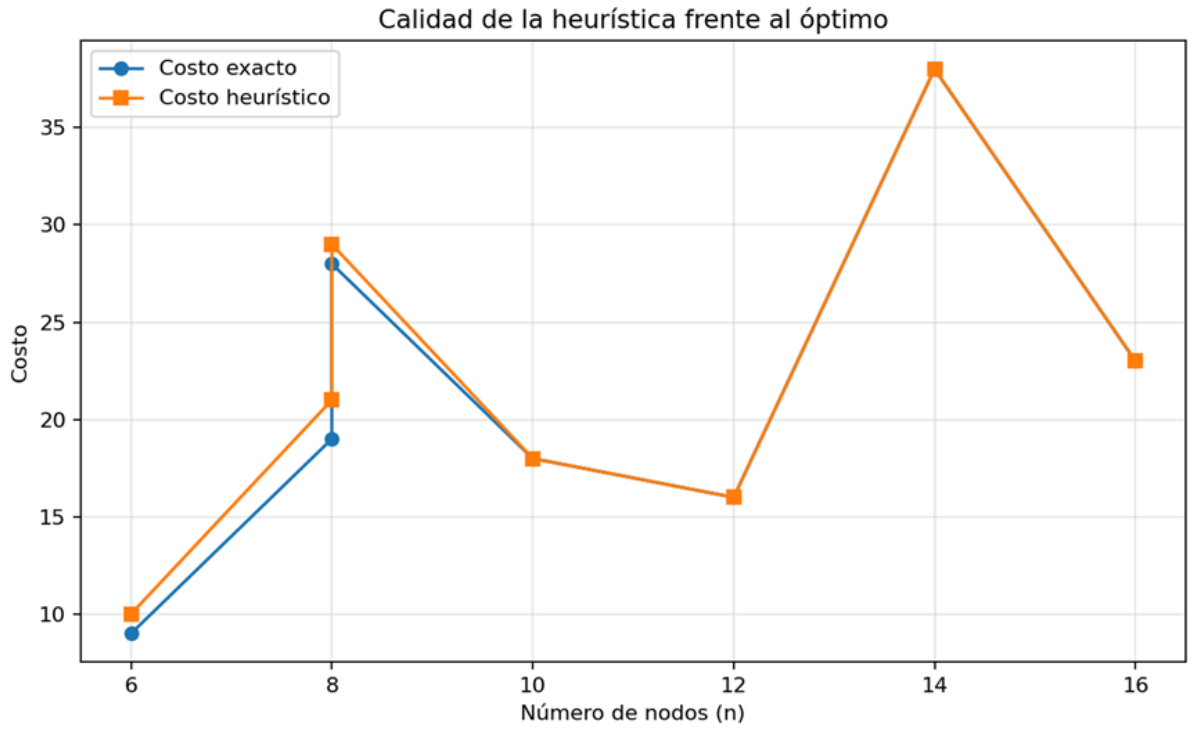


Figura 6.5. Comparación entre costo óptimo y costo heurístico.

6.2. Escalabilidad

A partir de los resultados obtenidos, se observa una diferencia clara en la escalabilidad entre ambos enfoques.

La solución exacta presenta un crecimiento exponencial en el tiempo de ejecución a medida que aumenta el número de nodos (nnn), lo que la hace inviable para instancias medianas o grandes. Esto se evidencia especialmente en Python y Java, donde los tiempos aumentan de forma significativa incluso con incrementos pequeños en el tamaño del grafo.

En contraste, la solución heurística muestra un comportamiento mucho más estable, manteniendo tiempos de ejecución bajos y creciendo de forma mucho más lenta. Esto indica que la heurística es altamente escalable y adecuada para trabajar con grafos de mayor tamaño.

6.3. Diferencias entre enfoques

Las diferencias entre la solución exacta y la heurística son claras tanto en rendimiento como en calidad de solución.

La solución exacta garantiza encontrar el costo mínimo óptimo, pero a un costo computacional muy alto debido a la exploración exhaustiva del espacio de soluciones. Esto limita su uso a instancias pequeñas.

Por otro lado, la solución heurística sacrifica la garantía de optimalidad, pero logra obtener soluciones muy cercanas al óptimo en tiempos significativamente menores. En los resultados observados, la heurística presenta pequeñas diferencias en algunos casos, pero en la mayoría logra igualar el valor óptimo.

6.4. Diferencias entre lenguajes

El análisis también muestra diferencias claras entre los lenguajes de programación utilizados.

C + + presenta el mejor rendimiento en todos los casos, con tiempos de ejecución significativamente menores tanto en la solución exacta como en la heurística. Esto se debe a su mayor eficiencia en el manejo de memoria y ejecución.

Java muestra un rendimiento intermedio, siendo más eficiente que Python pero generalmente más lento que C + +.

Python, si bien es más simple de implementar, presenta los mayores tiempos de ejecución, especialmente en la solución exacta, donde el crecimiento exponencial impacta más fuertemente.

7. Conclusión

En este trabajo se estudió el problema de **Steiner Tree**, abordando desde su definición hasta su implementación en Python, C + + y Java, utilizando tanto un enfoque exacto como uno heurístico.

Los resultados mostraron que la solución exacta garantiza la optimalidad, pero presenta un crecimiento exponencial que la hace inviable para instancias grandes. En contraste, la heurística demostró ser eficiente y escalable, obteniendo soluciones iguales o muy cercanas al óptimo en la mayoría de los casos.

Además, se observó que C + + ofrece el mejor rendimiento, seguido por Java y Python.

En resumen, el problema evidencia el compromiso entre precisión y eficiencia, siendo las heurísticas la alternativa más adecuada para aplicaciones prácticas de gran tamaño.

8. Bibliografía

- Karp, R. M. (1972). Reducibility among combinatorial problems. Complexity of Computer Computations.

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). Introduction to Algorithms.

- Hwang, F. K., Richards, D. S. y Winter, P. (1992). The Steiner Tree Problem.

9. Anexo

- codigo/python/steiner_tree.py
- codigo/cpp/steiner_tree.cpp
- codigo/java/SteinerTree.java
- codigo/casos/*.txt
- codigo/resultados/resultados_experimentales.csv